How to Price AI Solutions: The Complete Masterclass

This guide summarizes the key concepts, models, and steps from the video.

By: Nate Herk

The Pricing Principle That Changes Everything

The one rule to remember

Before you say any price, get the client to tell you what the problem is costing the business. Use the client's numbers, the client's language, and the client's definition of success. Once a credible value number is on the table, your fee becomes a fraction of a number the client has already stated.

> **Core principle:** Sell the result. The build is simply how the result gets delivered.

This is **client stated value**. It is more persuasive than a spreadsheet you invented after the call because the assumptions came from the person who owns the problem. Your job is to help the client make invisible costs visible, check the assumptions, and connect the proposed system to a measurable business result.

The full pricing sequence

1. Diagnose the business problem before discussing a solution.
2. Map the current process so hidden steps and exceptions become visible.
3. Quantify the credible first year value with the client.
4. Confirm the technical scope, delivery risk, and **cost floor**.
5. Choose a price inside the defensible corridor between cost and value.
6. Present distinct options that change scope and value.
7. Collect payment at signing and at **objective milestone**s.
8. Measure the result so this project makes the next project easier to sell.

What this guide helps you produce

- A discovery conversation that uncovers value without feeling pushy.
- A price that can be explained in one minute with client supplied numbers.
- A proposal with clear options, assumptions, milestones, and run costs.
- A payment schedule that limits unpaid work to roughly 30 days.

- A maintenance agreement that does not quietly become unlimited development.
- A proof system that converts delivery results into expansion work and case studies.

Why Hourly Billing Becomes the Wrong Incentive

Hourly billing rewards slowness

Imagine two developers billed to a client at $150 per hour. The strongest developer completes a feature in one day. The slowest developer needs three days. Hourly billing produces three times more revenue from the weaker performance. If the strongest developer becomes even faster, the provider earns even less.

That incentive becomes more distorted as AI tools compress a week of work into an afternoon. A provider should not lose income because the team developed better systems, reusable components, stronger judgment, and faster execution.

When hourly work can still make sense

Hourly work is not forbidden. It can be useful for a first project when you have little proof, for open ended troubleshooting, or for advisory work where the deliverable cannot be defined yet. Nate's practical starting point is that the first two or three projects can be easier to sell around $100 per hour.

- Set a cap so the client knows the maximum exposure.
- Define the learning deliverable, such as a process map, audit, or technical diagnosis.
- State what decision the client will be able to make when the hourly phase ends.
- Convert the implementation into a fixed price once scope and value are clear.

The better model after you have proof

Value based pricing connects the fee to the business outcome instead of the hours consumed. The client evaluates the investment against time recovered, errors avoided, capacity created, or revenue enabled. Your faster execution then improves margin without reducing the client's result.

> **Remember:** Cost does not justify price. Price justifies cost.

A landscaper cannot double the fee for the same lawn simply because a new truck increased the landscaper's expenses. The client did not receive more value. Your cost matters because it establishes the lowest responsible price, but it does not establish the client's **value ceiling**.

The Three Numbers: Cost, Value, and Price

Cost is your floor

The cost floor is the lowest price that makes the engagement responsible for your business. Below this number, you should reduce scope, change the delivery model, or walk away.

- Delivery labor, including design, development, testing, documentation, and training.
- Specialist costs, including security, legal, data, or industry expertise.
- Testing usage, temporary infrastructure, model calls, and evaluation data.
- Project management, meetings, revisions, and client communication.
- Contingency for known uncertainty and a reasonable profit margin.

> **Cost floor formula:** Delivery cost plus risk reserve plus required profit.

Value is the client's ceiling

The value ceiling is the most the solved problem could reasonably be worth to this specific business. It is not the largest exciting number you can imagine. It is the credible economic impact that the client can explain internally.

- Labor capacity recovered.
- Hard dollar losses avoided.
- Revenue capacity enabled.
- Hiring or contractor cost avoided.
- Cycle time reduced.
- Risk exposure reduced.
- Customer experience, retention, or response speed improved.
- Strategic capacity created for higher value work.

Price lives between the two

The **price corridor** is the space between your cost floor and the client's value ceiling. A healthy project needs enough room inside that corridor to absorb uncertainty, deliver professionally, and produce an obvious return for the client.

> **Decision test:** If the client's credible value is below your responsible cost, there is no deal at the current scope.

Do not solve a broken corridor by hiding your costs or inventing value. Change the scope. Start with a smaller outcome, reduce uncertainty with paid discovery, or recommend a simpler tool.

Discovery Comes Before the Number

Think like a doctor

A good discovery call is a diagnosis, not a performance. You are deciding whether a real problem exists, whether automation is the right intervention, and whether this is a project you can deliver responsibly.

Qualify the opportunity

Before going deep, test four signals.

- Conviction: Does the client genuinely want this problem solved?
- Urgency: What business pressure makes action important now?
- Authority: Who can approve the project, budget, security, and access?
- Strategic fit: Is the work aligned with your skills, proof, and preferred market?

Weak signals do not mean the person is a bad prospect. They mean the project may not be ready. Protect your time instead of creating a proposal for a conversation that has no owner, urgency, or decision path.

Use the LRP framework

The **LRP framework** means Listen, Repeat, Poke. Let the client describe the situation. Repeat the important part in plain language so both sides confirm the facts. Then poke with a specific question that reveals frequency, volume, cost, failure, or consequence.

- Listen: Let the client explain what has been tried, what worked, and where it gets stuck.
- Repeat: Reflect the problem using the client's own words.
- Poke: Ask how often it happens, how many people touch it, and what failure costs.

Run the three why questions

Why This Why Now Why Me is a compact way to test whether the proposed project and provider make sense.

- Why this: Does the requested AI solution actually create the desired outcome?
- Why now: What deadline, competitive pressure, cost, or growth constraint creates urgency?
- Why me: Why not build this internally, assign it to an intern, or buy existing software?

Ask the uncomfortable question directly. If the client says an internal person could build a version, agree that they probably could. Then ask who will harden it, monitor it, handle model changes, test edge cases, and scale it when volume grows. Stop and let the client answer. Their answer reveals the real reason they are considering outside help.

Delay price with confidence

> **Script:** I want to make sure I understand the technical landscape before I give you a number. Let me ask a few more questions so I do not misquote you.

A confident delay is professional. A random number is not. If the client will not allow enough discovery to understand the result and scope, they may be shopping only for the cheapest labor. That is usually a poor fit for a consulting relationship.

The Time Lens and the Income Lens

The Time Lens finds expensive manual work

The **Time Lens** reveals repeatable processes that consume human attention. Start broad, then narrow to a team, task, frequency, and cost.

- Which department is currently the most expensive in terms of manual labor?
- If your team gained five extra hours each week, what would they focus on?
- How long does this take today?
- How many people touch it?
- How often does it happen?
- What systems are involved?

The Income Lens finds growth constraints

The **Income Lens** reveals the process that prevents the business from handling more demand. This can be more valuable than saving hours because removing the constraint unlocks capacity.

- If your leads doubled tomorrow, what would break first?
- What process is currently limiting your ability to grow?
- Where are you turning away work, delaying customers, or adding headcount?
- What becomes possible if this bottleneck disappears?

Why you need both lenses

Time savings are usually the easiest value to calculate, but they can understate the opportunity. A 45 minute daily task can be worth far more than the labor if mistakes in that task create $12,000 per month in scheduling losses. Begin with hours, then inspect the bottom line impact around the entire process.

> **Discovery rule:** Count the hours, then ask what those hours, delays, errors, and constraints do to the business.

Map the Process Before You Scope the Build

The clean story is rarely the full process

Clients describe workflows from their own vantage point. An approval, exception path, second system, data cleanup step, or manual workaround often appears only after the project begins. A live process map exposes those missing pieces before they erase your margin.

Map four parts

1. Trigger: What event starts the process?
2. Steps: What happens in order, who acts, and which systems are used?
3. Decisions and exceptions: What changes the path, and what happens when data is incomplete?
4. Result: What observable business output marks the process as complete?

Point to the constraint

Do not assume the client sees the bottleneck. Name the exact step, its time, its frequency, and its consequence. Then show the changed process in plain language. The client should be able to see what humans still handle and what the system takes over.

- Current state: A coordinator checks every application for 30 minutes and creates an afternoon backlog.
- Future state: The system applies agreed criteria in seconds and sends only exceptions to the coordinator.

Clarify the technical ecosystem

- Systems: What platforms, databases, spreadsheets, inboxes, and internal tools are involved?
- Access: Are reliable APIs, exports, webhooks, or service accounts available?
- Data: Is the input structured, complete, consistent, and legally usable?
- Scale: What is normal volume, peak volume, and expected growth?
- Security: What privacy, compliance, retention, or deployment constraints apply?
- Users: Who must review, approve, correct, or override the system?
- Change: Which current process variations are likely to appear after launch?

> **Readiness check:** If you cannot name the specific outcome and the major technical constraints, you are not ready to quote.

Build the First Year Value Model

Start with annual labor capacity

Annual labor value: Volume per period x time per item x **loaded labor rate** x periods per year.

Use loaded labor rate, not only salary. Loaded rate can include wages, payroll costs, benefits, management overhead, and other employment costs. If the client does not know the exact rate, use a conservative range and label the assumption.

Calculate avoided error cost

Annual avoided error value: Incidents per month x average cost per incident x 12 x expected reduction.

Error cost can include refunds, rework, missed appointments, scheduling failures, penalties, churn, rush shipping, or customer recovery time. Use documented history when possible. Do not treat every possible incident as guaranteed.

Calculate revenue capacity

Annual revenue capacity: Additional qualified opportunities x contribution per outcome x realistic conversion rate.

Use contribution, not gross revenue, when margins are meaningful. If an appointment is worth $5,000 in revenue but only $2,000 in contribution, the second number is the stronger basis for pricing. Separate capacity from guaranteed revenue. The system can enable a result without controlling every factor needed to produce it.

Calculate hiring cost avoided

Hiring value: Avoided annual loaded cost plus recruiting, onboarding, and management cost.

Be precise with the language. Recovered capacity does not always mean the company will eliminate a role. It may mean the same team can handle more volume, improve service, or postpone a hire. Present the benefit that is actually expected.

Calculate cycle time value

Faster response, onboarding, quoting, or reporting can improve conversion, retention, and cash flow. Quantify only the part the client can defend. A strong model might value the labor savings directly and present improved conversion as upside rather than guaranteed value.

Add confidence adjustments

A **confidence adjustment** keeps projections honest. Give each value component a confidence level based on the quality of the baseline and your control over the outcome.

- High confidence: Direct labor time with reliable volume and rate data.

- Medium confidence: Avoided errors supported by several months of records.
- Lower confidence: Future revenue that depends on sales performance, adoption, or market demand.

Confidence adjusted value: Estimated value x confidence percentage.

Example: A projected $60,000 revenue benefit at 50% confidence contributes $30,000 to the conservative model. Show the unadjusted upside separately if useful.

Prevent double counting

Do not add two value lines that describe the same benefit. If recovered staff time is used to handle more customers, you may be measuring the same capacity once as labor and again as revenue. Choose the most defensible primary model and treat the other as supporting context.

- Ask whether each value line can happen independently.
- Use conservative assumptions when benefits overlap.
- Create a base case and an upside case instead of stacking every possibility.
- Have the client confirm the final model in writing.

Choose a Defensible Price

Use 10% to 20% as a starting point

Nate's practical starting range is 10% to 20% of credible first year value. This is a starting point, not a law. The final number must still clear your cost floor and reflect scope, risk, proof, competition, and the client's alternatives.

Starting price: Credible first year value x 10% to 20%.

Run the 10X check

The **10X check** asks whether the client can reasonably see ten dollars of first year value for every dollar invested. That ratio makes the decision easier, but it is not a guarantee. Use a conservative base case to support the price and present projections as projections.

Value multiple: Credible first year value divided by project price.

- 10X or greater: The economics are usually easy to explain.
- 5X to 10X: The project may still be strong if the value is reliable and strategic.
- Below 5X: Revisit scope, price, value evidence, or project sequence.

Use proof to move within the corridor

A provider with no case studies should not expect the same share of value as a specialist with repeated proof, strong risk controls, and a delivery team. As your evidence improves, you can capture more of the value without changing the client's return.

- Early proof: Use a smaller project, narrower scope, or easier commercial structure.
- Relevant proof: Price with more confidence when the case study closely matches the industry and process.
- Enterprise proof: Charge for governance, security, stakeholder management, documentation, and lower execution risk.

Use a margin check

Value establishes the ceiling, but your business still needs a responsible floor. One useful internal check is the gross margin formula.

> **Required price:** Estimated delivery cost divided by one minus target gross margin.

Example: If delivery is expected to cost $6,000 and the target gross margin is 60%, the required price is $15,000. If the client's credible value supports only $10,000, change the project. Do not accept a price that makes professional delivery impossible.

Price Complexity, Risk, and Testing

Complexity changes the cost floor

Two systems that create the same business value can require very different delivery effort. Score the following before finalizing price.

- Number and quality of integrations.
- Structured versus unstructured input.
- Data cleanup and migration.
- Number of user roles and approval paths.
- Exception volume and business rule complexity.
- Peak throughput and latency requirements.
- Security, compliance, and deployment constraints.
- Documentation, training, and change management.
- Stakeholder count and approval process.

Autonomy increases testing

The more freedom an AI system has to act, the more rigorous evaluation and quality assurance must become. A deterministic notification workflow is cheaper to validate than an agent that communicates with customers, updates records, or takes actions across multiple tools.

- Define representative test cases.
- Include normal cases, edge cases, and adversarial inputs.
- Test failure handling, retries, escalation, and human override.
- Measure accuracy, latency, cost, and completion rate where relevant.
- Document the launch bar and the rollback plan.

Include testing in your build price

Testing costs can range from roughly $100 to several thousand dollars depending on model usage, autonomy, data volume, and evaluation rigor. Nate's practical approach was to add roughly $1,000, $2,000, or $3,000 to a project when that matched the expected testing burden.

Treat testing as a real delivery cost. Before production, the provider can cover test usage and include it in the fixed fee. At production, replace test credentials with client owned accounts and billing unless security requirements dictate a different arrangement.

Use Options Instead of One Number

Options change the buying decision

One price asks the client whether to work with you. Three credible options ask how to work with you. The client compares scope, speed, support, and value instead of comparing your single number with a cheaper vendor.

Design an option architecture

Option architecture means each package solves a different sized problem. Do not place the same outcome behind three arbitrary prices. Every step up must create additional client value or reduce meaningful risk.

- Essential: The smallest complete outcome and a narrow deployment.
- Growth: The recommended outcome with broader integrations, testing, and adoption support.
- Scale: Wider business coverage, stronger governance, faster response, or a phased roadmap.

Use the middle option intentionally

The middle option is often designed to win because it balances the result, risk, and investment. The lower option should be genuinely useful, not intentionally broken. The upper option should serve a real buyer who values broader scope, speed, governance, or access.

A practical value based example

If credible first year value is $100,000, an illustrative structure could begin around $10,000, $25,000, and $50,000. Those percentages are not a template to copy blindly. The $25,000 and $50,000 options require more value, not merely more technical features.

- Essential at $10,000: One priority workflow, one team, core integrations, standard launch support.
- Growth at $25,000: Multiple connected workflows, stronger evaluation, training, and 60 day optimization.
- Scale at $50,000: Broader process redesign, governance, multiple teams, and an execution roadmap.

Write the proposal in the right order

1. Current state in the client's words and numbers.
2. Desired future state and business result.
3. Why your experience and approach fit this engagement.
4. Options with outcomes, scope, assumptions, and exclusions.
5. Objective milestones and payment events.
6. Estimated production run cost and volume assumption.
7. Measurement plan, support model, and commercial terms.

Structure Payments Around Objective Milestones

Limit your unpaid exposure

A healthy payment schedule pays the provider at signing and then roughly every 30 days. The goal is to avoid carrying months of unpaid work while the client debates whether an ambiguous milestone feels complete.

Example for a $9,000 project

- $3,000 at signing to reserve capacity and begin discovery.
- $3,000 when the proof of concept milestone passes.
- $3,000 when the production milestone passes and handoff begins.

The percentages can change. The principle is that cash collection follows real progress and no payment depends on a vague emotional judgment.

Write objective milestones

An objective milestone is observable, testable, and difficult to argue with. It describes the artifact, the input, the behavior, the environment, and the evidence required to mark the milestone complete.

> **Objective example:** A proof of concept is available to the business owner. When the owner submits a question, the system retrieves information from the agreed database and returns a response within one minute.

This milestone does not claim the database is perfectly optimized or every answer is correct. Those may be later acceptance criteria. It states exactly what can be tested now.

> **Subjective example to avoid:** The inbox agent is working as expected.

The milestone quality checklist

- Artifact: What exists at this point?
- User: Who can access and test it?
- Input: What exact event or prompt starts the test?
- Behavior: What must the system do?
- Threshold: What measurable bar must it meet?
- Evidence: What log, recording, report, or test result proves completion?
- Exclusions: What is intentionally reserved for a later milestone?

Protect the engagement contractually

Use a **Master Services Agreement** for stable relationship terms and a **Scope of Work** for project specific scope, milestones, fees, and assumptions. Define the communication channel, response window, client responsibilities, access deadlines, intellectual property ownership, and what happens if the client is unresponsive for 30 days.

Contract language should be reviewed by a qualified attorney in the relevant jurisdiction. Community templates are a starting point, not a substitute for professional advice.

Control Scope Without Killing Momentum

Treat new ideas as positive signals

Scope creep often means the client is excited and imagining more value. Do not punish that enthusiasm, and do not quietly absorb the work.

> **Script:** That is a great idea, and I can see how it would add value. Let us add it to the version two backlog so we keep the current milestones moving and deliver the value we already agreed on as quickly as possible.

Use a change request

A **change request** documents new functionality, the reason for the change, affected milestones, additional price, timeline impact, and approval. It keeps the original agreement intact while giving the client a clean path to expand.

- New integration.
- New user group or business unit.
- New decision logic.
- New output or communication channel.
- Materially higher volume.
- New compliance or security requirement.
- New feature that was not part of the acceptance criteria.

When the client says the price is too high

Do not teach the client that your price falls every time they frown. Reduce scope, risk, or speed.

> **Script:** It sounds like $20,000 is not in the budget right now. Let us reduce the scope and start with the highest value piece. Once that is working and returning time or capacity, we can move to the next piece together.

- Keep the same price for the same scope.
- Reduce scope to fit the budget.
- Sequence the project into smaller phases.
- Replace custom development with existing software when appropriate.
- Use paid discovery to reduce uncertainty before implementation.

Separate Maintenance From New Functionality

What maintenance means

A **maintenance retainer** keeps the agreed system performing the agreed function. It is not an unlimited bucket for new features.

- Monitoring health and error logs.
- Repairing failures caused by API or model changes.
- Handling agreed edge cases that prevent scoped behavior.

- Small configuration adjustments needed to preserve the original function.
- Routine checks of volume, latency, and operational cost.

What maintenance does not mean

- New workflows.
- New integrations.
- New user experiences.
- New business rules.
- New reports or outputs.
- Material changes in volume or compliance.
- Ongoing implementation of a broader roadmap.

How to price maintenance

Nate used a simple $400 per month maintenance package at one stage of his agency. The number worked because the automations were designed to require limited attention. Do not copy $400 when expected support cost, risk, or response obligations are higher.

- Estimate the normal monthly monitoring and repair cost.
- Add the cost of the response window you promise.
- Account for system criticality and the impact of downtime.
- State included hours or included activities if that prevents ambiguity.
- Price premium support separately when the client requires faster response.

Maintenance versus roadmap retainer

A **roadmap retainer** is a different product. It can include three components: implementing the next approved improvement, maintaining what is already live, and identifying new opportunities. The client is buying steady execution of a phased plan, not only system health.

> **Retainer framing:** The improved business is the product. The monthly structure is how the roadmap gets delivered.

Put Production Costs in the Client's Name

Client account, client card

API usage, cloud subscriptions, tokens, telephony, storage, and other production utilities should normally sit in the client's account and on the client's payment method. Your fee covers design, consulting, building, testing, documentation, and support.

This makes ownership and usage transparent. It also prevents the provider from becoming a billing intermediary who must chase reimbursements, explain blended charges, and carry the client's utility expenses.

Estimate run cost in the proposal

Give the client an expected monthly production cost with the volume assumption next to it. Label it as an estimate, not a guarantee.

> **Example:** Estimated monthly model and infrastructure cost is $180 at 12,000 processed items. Cost will change with volume, model selection, and provider pricing.

Move credentials at production

During testing, the provider can pay for temporary usage and include it in the fixed project price. At production, use client owned credentials and billing. For regulated or enterprise environments, the client may require all testing to occur inside an approved account from the beginning.

The Appointment Setting Agent Case Study

The baseline

The business had employees manually setting appointments for about 20 leads each week. Each lead consumed roughly one hour of human time. The loaded labor cost was about $40 per hour.

> **Weekly labor value:** 20 leads x 1 hour x $40 = $800.
>
> **Annual labor value:** $800 x 52 weeks = $41,600.

The build price

The project was priced at $5,500, roughly 13% of the annual labor value.

> **Value multiple:** $41,600 divided by $5,500 = about 7.6X.

The base labor case did not reach the preferred 10X check. Additional upside could come from faster response, better lead quality, and the ability to handle rising lead volume. That upside should be framed as a projection, not a guarantee.

The ongoing fee

The maintenance plan was $400 per month. It covered keeping the scoped system working when APIs, models, or edge cases changed. It did not include new functionality.

The missed opportunity

The baseline was captured, but the after state was not measured consistently. Without a 30 day, 60 day, and 90 day transformation record, the project created less proof for the next sale than it could have.

How to improve this proposal today

- Track weekly lead volume, time per lead, speed to lead, meeting completion, and exception rate.
- Separate guaranteed labor capacity from projected conversion upside.
- Define the exact production acceptance criteria.
- State the $400 maintenance boundary in writing.
- Schedule the 30 day success review before launch.
- Use the verified result to propose the next highest value bottleneck.

Four More Pricing Scenarios

Scenario 1: Construction order capture

A simple system converted daily phone orders into the text format a construction crew already used. It saved only about 45 minutes per day, but it helped avoid roughly $12,000 per month in scheduling errors.

The hard dollar loss is the stronger value anchor. If you priced only the labor time, you would miss the reason the project matters.

- Primary value: Avoided scheduling losses.
- Secondary value: Recovered administrative time.
- Proof metric: Monthly error cost before and after launch.
- Pricing lesson: Simple technical work can create high business value.

Scenario 2: Property maintenance routing

An admin team handles about 40 maintenance requests each week. Each request takes 15 to 20 minutes to read, categorize, assign, and follow up. The process consumes roughly 12 hours per week and peaks at 60 to 70 requests during busy periods.

- Labor value: 12 hours x loaded rate x 52 weeks.
- Capacity value: Ability to handle seasonal spikes without backlog.
- Complexity factors: Unstructured descriptions, routing logic, vendor communication, and platform access.
- Proof metrics: Routing time, backlog size, wrong assignments, and completion time.

Price after confirming the platform API, peak volume, exception rules, and the cost of delayed maintenance.

Scenario 3: Customer support automation

A five person support team handles about 200 tickets per day. Roughly half are routine. The team spends about four hours per day on repetitive responses. A well scoped system routes common questions automatically and gives complex tickets to humans with context.

- Baseline: Four hours per day of routine work.
- After state target: About 30 minutes per day of review.
- Monthly capacity: Roughly 70 hours recovered.
- At $30 per hour: About $2,100 monthly labor capacity.
- Additional value: Faster response and more attention for complex customers.

Annual labor capacity is about $25,200. A starting range at 10% to 20% is about $2,520 to $5,040 before checking integrations, risk, testing, and cost floor.

Scenario 4: Customer onboarding

A SaaS team onboards 15 to 20 customers per month. The baseline is 2.5 hours of manual work per customer, roughly 40 hours per month. Welcome messages take 18 hours, training takes six days to schedule, and follow up completion is about 80%.

- After 30 days: Human review drops to about 20 minutes per customer.
- Training is scheduled within one day.
- Follow up completion reaches 100%.
- Value includes labor capacity, faster customer activation, and fewer missed steps.

Price from the credible first year base case, then show faster activation and retention as supporting upside unless the client has reliable data connecting them to revenue.

How to Create a Three Option Proposal

Option 1: Essential

- One priority workflow.
- One user group.
- Core integration path.
- Standard evaluation set.
- Documentation and launch handoff.
- 30 day success review.

Option 2: Growth

- Everything in Essential.
- Additional integration or connected workflow.
- Larger evaluation set and edge case coverage.
- Team training and adoption support.
- 60 day optimization window.
- Recommended maintenance plan.

Option 3: Scale

- Everything in Growth.
- Multiple teams or business units.
- Governance, permissions, and reporting.
- Higher volume testing.
- Executive result dashboard.
- Phased improvement roadmap.
- Priority response agreement.

What belongs under every option

- Business result.
- Included scope.
- Explicit exclusions.
- Client responsibilities.
- Assumptions.
- Milestones and acceptance criteria.
- Payment schedule.
- Estimated run cost.
- Measurement plan.
- Support model.

The Discovery Call Script

Open the conversation

> **Script:** I would like to understand how this works today, where it gets stuck, and what changes if we solve it. If it looks like a strong fit, I can map the next step and then give you a number based on the real scope.

Let the client unload the context

- Tell me everything you know about this process.
- What have you tried already?
- What worked?
- Where does it keep getting stuck?
- Who experiences the problem most directly?

Quantify the current state

- How long does one cycle take?
- How many cycles happen each week or month?
- How many people touch it?
- What is a reasonable loaded rate for that team?
- What happens when it goes wrong?
- How often does failure happen?
- What does one failure cost?

Quantify the future state

- What is the best realistic outcome after this is live?
- What should become faster, cheaper, safer, or more reliable?
- What will the team do with the capacity it gets back?
- What metric would make leadership call this a success?
- When should we measure that metric?

Test urgency and alternatives

- Why is this important now?
- What happens if nothing changes for another year?
- Why not buy an existing tool?
- Why not build it internally?
- Who else needs to approve this?

Confirm the value in their words

> **Script:** Let me make sure I have this right. At the current volume, this is costing about $X each year before we include the error and capacity impact. Does that match how you see it?

Move to the next step

> **Script:** Based on what we mapped, the main constraint is X. The proposed system changes it by Y, while your team continues to handle Z. Would you like me to build this for you?

If the answer is yes, schedule the proposal review before the discovery call ends. Do not send a complex proposal into silence and hope it closes itself.

The Pricing Worksheet

Step 1: Capture the baseline

1. Process name and owner.
2. Volume per day, week, or month.
3. Minutes or hours per item.
4. People involved.
5. Loaded labor rate.
6. Failure frequency.
7. Cost per failure.
8. Current cycle time.
9. Current capacity limit.
10. Current software and infrastructure cost.

Step 2: Calculate value

- Annual labor capacity.
- Annual avoided error cost.
- Annual contribution enabled.
- Annual hiring cost postponed or avoided.
- Annual risk reduction that can be credibly estimated.
- Base case value after confidence adjustments.
- Upside case shown separately.

Step 3: Calculate the floor

- Expected delivery labor.
- Testing and evaluation.
- Specialists and contractors.
- Project management and client communication.
- Temporary tools and infrastructure.
- Risk reserve.
- Required profit.

Step 4: Set the corridor

- Responsible cost floor.
- Credible value ceiling.
- Starting range at 10% to 20% of first year value.
- 10X check.
- Margin check.
- Decision to proceed, reduce scope, or run paid discovery.

Step 5: Design options

- Smallest complete outcome.
- Recommended outcome.
- Broader strategic outcome.
- Distinct value and risk reduction in each option.

Step 6: Design payment

- Payment at signing.
- Payment at each objective milestone.
- No more than about 30 days between payment events.
- Clear treatment of delays caused by missing client access or feedback.

The Proposal Quality Checklist

Business case

- The proposal begins with the client's current state.
- Client supplied numbers and assumptions are visible.
- The desired outcome is measurable.
- Guaranteed scope is separated from projected upside.

Scope

- Systems, data, users, volume, and environments are named.
- Included functionality is explicit.
- Exclusions are explicit.
- Client responsibilities and access deadlines are explicit.
- Change request rules are explicit.

Commercial terms

- Each option has a clear outcome and price.
- Payment dates or milestone events are clear.
- Production utilities are assigned to the client.
- Estimated run cost includes a volume assumption.
- Maintenance and new functionality are separated.

Acceptance

- Every milestone can be tested.
- Success thresholds are objective.
- Evidence required for completion is named.
- Response windows and communication channels are named.
- Unresponsive client and pause rules are defined.

Proof

- Baseline metrics are recorded before the build.
- Launch, 30 day, and 60 to 90 day reviews are scheduled.
- The client knows how results will be measured.
- Case study and testimonial permissions are discussed at the right time.

Measure the Result and Build the Proof Loop

Capture the baseline before launch

Teams adapt quickly. Within weeks, the new workflow feels normal and the old pain becomes hard to remember. Capture how long, how many people, how often, and what goes wrong while the manual process is still visible.

Use a Results Tracker

A **Results Tracker** should include Metric, Baseline, Launch, 30 Days, and 60 to 90 Days. Choose three to five metrics that describe both operational health and business impact.

- Time per item.
- Volume completed.
- Cycle time.
- Error or escalation rate.
- Response time.

- Completion rate.
- Human review time.
- Cost per completed item.

Measure at three checkpoints

1. Launch: Confirm that the system works and record early operational data.
2. 30 days: Measure the real improvement after the team has adjusted.
3. 60 to 90 days: Confirm whether the improvement holds as usage and volume normalize.

Run a success review

1. Revisit the original problem.
2. Present the before and after numbers.
3. Translate each metric into business impact.
4. Ask the client whether the data matches the team's experience.

This **proof loop** makes value visible. It supports renewals, expansion projects, referrals, testimonials, and stronger pricing on the next engagement.

Turn the result into a case study

- Problem: What the process looked like before.
- Solution: What the system does in plain business language.
- Results: The measured before and after numbers.
- ROI: What those results mean in time, money, capacity, or risk.

Common Pricing Mistakes

Quoting before discovery

A fast guess feels helpful in the moment and becomes expensive later. Use the confident delay and earn the right to quote.

Pricing only from hours

Internal effort matters to your floor. It does not define the client's value. Lead with the result.

Counting only labor

Time is the easiest starting point. Always inspect errors, delay, capacity, retention, contribution, and risk.

Using gross revenue as guaranteed value

Use contribution where possible and reduce uncertain projections with confidence adjustments.

Double counting benefits

Do not count the same recovered capacity as both labor savings and new revenue unless the two outcomes can happen independently.

Selling one vague package

Give the client distinct options and define exactly how scope and value change.

Writing subjective milestones

Expected, intuitive, user friendly, and complete are arguments waiting to happen unless each term has a measurable definition.

Discounting instead of reducing scope

A lower price for the same scope teaches the client that your numbers are flexible. Offer a smaller complete result.

Calling unlimited improvements maintenance

Maintenance preserves agreed behavior. New functionality belongs in a new scope or roadmap retainer.

Paying production utilities for the client

Use client owned accounts and cards for production. Estimate usage and keep billing transparent.

Skipping the after state

A successful project without measured proof is a weaker asset than it should be. Schedule the result reviews before launch.

Automating a process that does not exist

If the workflow is unclear, capture the standard operating procedure first. In regulated or complex environments, a text based process and measurable KPI can be phase one. Build the interface only after the process works.

Ignoring simpler alternatives

Recommend existing software when it solves the problem responsibly. A buy first recommendation can create less immediate revenue and more long term trust.

Advanced Decisions for Experienced Providers

Paid discovery when uncertainty is high

Use a paid audit or discovery phase when the business case is promising but the process, data, security, or integration risk cannot be scoped responsibly. The deliverable can be a process map, opportunity matrix, technical findings, phased roadmap, value model, and implementation options.

Prioritize impact versus effort

Plot opportunities by impact and effort. Begin with high impact, lower effort work that proves value and builds context. Sequence higher effort systems after the client has evidence and the provider understands the business.

- Phase 1: Quick wins with clear metrics.
- Phase 2: Connected systems with more dependencies.
- Phase 3: Broader transformation, governance, and optimization.

Price governance and adoption

Enterprise clients are not paying only for code. Stakeholder alignment, security review, data governance, training, documentation, access controls, auditability, and change management are part of the delivery cost and part of the risk reduction.

Know when to walk away

- The client refuses sufficient discovery but demands a fixed quote.
- The value ceiling does not clear the cost floor.
- The client expects guaranteed revenue you cannot control.
- The required expertise is missing and cannot be added responsibly.
- Security or compliance requirements are unclear.
- The client wants unlimited scope inside a fixed budget.
- The decision owner, access owner, or success owner is missing.

A One Minute Price Explanation

The structure

1. Restate the problem and baseline.
2. State the conservative first year value.
3. State the proposed price.
4. Show the value multiple.

5. Name the major scope and risk assumptions.
6. Pause and invite questions.

Example

> **Script:** Your team is processing about 20 leads each week, at one hour per lead and a loaded rate of $40. That is about $41,600 in annual labor capacity. The build is $5,500, which is about 13% of that base value and gives you roughly 7.6 times the investment before we count faster response or additional volume. The price assumes the systems and access we reviewed, the agreed evaluation set, and the milestone schedule in the proposal.

A strong explanation is calm and mathematical. You are not defending a random fee. You are showing the client how their own facts became a commercial decision.

Your Next Client Pricing Workflow

Before the call

- Prepare the qualification checklist.
- Prepare the Time Lens and Income Lens questions.
- Open a blank process map.
- Open the pricing worksheet.
- Know your cost floor inputs and target margin.

During the call

- Listen, repeat, and poke.
- Map the process.
- Quantify current cost and desired result.
- Clarify systems, data, users, volume, and risk.
- Confirm the value model in the client's language.
- Book the proposal review live.

Before the proposal

- Calculate base case and upside value.
- Check for double counting.
- Calculate cost floor and margin.
- Design three complete options.
- Write objective milestones.

- Estimate production run costs.
- Schedule payment events no more than about 30 days apart.

After signature

- Capture the baseline.
- Collect access and client responsibilities.
- Run evaluation against the agreed bar.
- Invoice at objective milestones.
- Move production utilities to client owned accounts.
- Run the 30 day success review.
- Document the result and identify the next bottleneck.

Key Takeaways

1. Get the client to state what the problem costs before you state a price.
2. Keep cost, value, and price separate. Cost is your floor, value is the client's ceiling, and price lives between them.
3. Use the Time Lens for manual effort and the Income Lens for growth constraints.
4. Start around 10% to 20% of credible first year value, then check proof, risk, margin, and the 10X logic.
5. Present distinct options and reduce scope instead of discounting the same work.
6. Tie payments to objective milestones and avoid carrying more than roughly 30 days of unpaid work.
7. Keep maintenance, new functionality, roadmap work, and production utilities commercially separate.
8. Measure the baseline, 30 day result, and 60 to 90 day result so every project builds proof for the next one.

Want to connect with others building and monetizing AI automation?

[Become an AIS Plus Member]